NEXUS

Agentic AI Productivity Assistant

Full implementation roadmap • Architecture • Technology stack • Milestones • Security • Deployment

Project type	Production-style hybrid voice + agentic AI platform
Primary interface	Natural voice conversation and web dashboard
Backend	Python 3.12, FastAPI, LangGraph
LLM and voice	OpenAI API, Realtime voice, modular STT → LLM → TTS
Data layer	PostgreSQL + pgvector, Redis
Desktop target	Windows 11 local companion agent

Build philosophy

Build in verified milestones. Complete and test each phase before moving to the next. The plan is designed for a hands-on implementation, with the local Windows agent separated from cloud services and sensitive actions protected by explicit user approval.

Estimated schedule: approximately 46 working days at 5–6 focused hours per day. Actual timing depends on API access, integration complexity, testing, and deployment choices.

1. Product vision and scope

Nexus is a hybrid Agentic AI productivity assistant that lets a user speak naturally, delegate multi-step tasks, and manage work through connected services and an authenticated Windows desktop companion. The central orchestrator interprets requests, plans actions, routes work to specialist agents, applies permission checks, and reports verified results.

Core capabilities

- **Voice assistant:** natural conversation, speech recognition, speech generation, interruption handling, voice activity detection, and turn management.
- **Agent orchestration:** intent classification, planning, tool selection, multi-step execution, retries, validation, and recovery.
- **Task management:** create, update, prioritize, complete, and summarize tasks.
- **Email assistant:** search and summarize Gmail messages, prepare drafts, and send only after explicit approval.
- **Calendar assistant:** inspect events and availability, propose schedules, detect conflicts, and create or modify events with approval.
- **Desktop companion:** perform allowlisted Windows actions, open approved apps and websites, and search permitted folders.
- **Reminders and memory:** schedule reminders, retain useful context, and retrieve relevant memories.
- **Dashboard and evaluation:** inspect tasks, approvals, activity, agent performance, latency, errors, and usage.

Out of scope for the first release

- Unrestricted remote control of the computer or arbitrary shell execution.
- Sending emails, deleting files, or creating calendar events without a clear approval step.
- Storing passwords, API keys, or unnecessary sensitive message contents in conversation logs.

2. System architecture

The React dashboard and voice client communicate with a FastAPI backend. The backend manages identity, sessions, APIs, permissions, and persistence. Voice traffic is handled by a dedicated voice layer, while LangGraph coordinates specialist agents and tools. The Windows companion connects outbound to the backend using an authenticated channel and validates every received action locally.

Layer	Responsibilities
React + TypeScript UI	Dashboard, chat, voice controls, tasks, email, calendar, approvals, activity, settings
FastAPI backend	Authentication, REST APIs, WebSockets, session management, authorization
Voice engine	Realtime speech-to-speech and modular STT → LLM → TTS provider interface
LangGraph orchestrator	Intent, plan, route, execute tools, approval gates, validate, summarize
Specialist agents	Tasks, email, calendar, reminders, memory, desktop
Data + jobs	PostgreSQL/pgvector, Redis, background worker, audit trail
Windows companion	Authenticated outbound connection; locally validates allowlisted desktop actions

Key design rule

The LLM may propose an action, but deterministic backend policy decides whether it is allowed and whether approval is required. The local desktop agent must validate commands independently; it must never execute arbitrary instructions generated by a model.

3. Technology stack

Area	Recommended technology
Language/runtime	Python 3.12; Node.js 22 LTS
Frontend	React, TypeScript, Vite, Tailwind CSS
Backend	FastAPI, Uvicorn, Pydantic, SQLAlchemy
Agent orchestration	LangGraph; custom Python tools; LangChain where useful
LLM	OpenAI API using a configurable, currently supported model
Voice	OpenAI Realtime API plus interchangeable STT and TTS adapters
Database	PostgreSQL + pgvector; Alembic migrations
Background work	Redis + Celery or an equivalent task worker
Google integrations	Gmail API, Google Calendar API, OAuth 2.0
Desktop automation	PyWinAuto, PyAutoGUI, Playwright; strict command allowlists
Testing	Pytest, Vitest, integration and agent-evaluation tests
Deployment	Docker Compose locally; Vercel frontend; cloud backend/database/Redis

Model configuration note: model identifiers and API capabilities change. Keep the model name in environment configuration and verify it against current OpenAI API documentation and the models available to your API account before making live requests. A ChatGPT subscription does not automatically include API usage or API credits.

4. Main agents and workflows

The orchestrator should expose specialist capabilities through narrowly scoped tools. Each tool should have typed inputs, typed outputs, permission checks, timeouts, and clear error handling.

- **Task agent:** task CRUD, priority and due-date updates, completion, and daily summaries.
- **Email agent:** Gmail search, message retrieval, summarization, draft generation, and approval-gated send.
- **Calendar agent:** event listing, availability checks, conflict detection, proposed changes, and approval-gated writes.
- **Reminder agent:** one-time and recurring reminders, scheduled execution, and notifications.
- **Memory agent:** conversation context, user-approved preferences, semantic retrieval, and user-controlled correction/deletion.
- **Desktop agent:** allowlisted application/browser actions and constrained file operations on the paired Windows device.

Example multi-agent workflow

1. User says: “Find the latest interview invitation, check my availability tomorrow, and prepare a calendar event.”
2. Email agent searches relevant messages and extracts the proposed meeting details.
3. Calendar agent checks the user's calendar for conflicts.
4. Orchestrator asks the user to clarify any missing details and presents the proposed event.
5. User approves the event creation.

6 Calendar tool creates the event; the backend verifies the result and records an audit entry.

7 Assistant confirms what was actually completed.

5. Orchestration design

A practical initial LangGraph workflow can use the following nodes. Keep state explicit and persist execution checkpoints for recoverability.

1. **understand_request:** normalize the message and conversation context.
2. **classify_intent:** identify the primary task and any required specialist agents.
3. **plan_actions:** produce a structured plan with tool arguments and dependencies.
4. **route_agent:** select the appropriate agent or sequence of agents.
5. **execute_tool:** call a narrowly scoped backend tool.
6. **approval_gate:** pause the workflow when policy requires user consent.
7. **validate_result:** verify the external system's response and detect partial failure.
8. **save_memory:** persist only appropriate, useful context.
9. **generate_response:** explain the outcome, remaining steps, and any errors.

Suggested graph state fields

- user_id, conversation_id, and request_id
- user_message, normalized_intent, and plan
- current_agent, tool_calls, and tool_results
- approval_required, approval_status, and approval_payload
- workflow_status, error_details, and final_response

6. Database design

Use UUID primary keys, UTC timezone-aware timestamps, foreign keys, indexes, and strict user-level data isolation. Use Alembic for schema migrations. Keep OAuth tokens encrypted at rest and never write secrets to application logs.

Domain	Tables
Identity and conversations	users, oauth_accounts, conversations, messages
Productivity	tasks, reminders, calendar_events, email_drafts
Agent operations	agent_runs, tool_executions, approvals
Desktop	desktop_devices, desktop_actions
Memory and governance	memories, audit_logs, voice_sessions

For semantic memory, store the embedding model name and dimension alongside the design decision. The pgvector column dimension must match the embedding model. Do not assume all embedding models use the same vector size.

7. API surface

Endpoint family	Purpose
/health	Health and dependency checks
/api/v1/chat	Submit a chat request and retrieve assistant response

Endpoint family	Purpose
/api/v1/conversations	Create and retrieve conversations
/api/v1/voice/session	Create voice session
/ws/voice/{session_id}	Voice streaming channel
/api/v1/tasks	Task CRUD and completion
/api/v1/emails	Search and summarize email
/api/v1/emails/drafts	Create and manage drafts
/api/v1/calendar/events	List and manage calendar events
/api/v1/calendar/availability	Check availability
/api/v1/reminders	Create and manage reminders
/api/v1/approvals	List, approve, or reject pending actions
/api/v1/desktop	Pair device, status, and validated actions
/api/v1/memory	Inspect and manage user memory
/api/v1/analytics	Workflow and quality metrics

8. Implementation roadmap

The project is divided into ten milestones. Each phase has a concrete deliverable and a verification checkpoint. Do not advance until the current milestone works on your machine.

Phase 1 — Foundation (Days 1–3)

Build: Set up Python, Node, Git, Docker, VS Code; create backend and frontend; configure PostgreSQL + pgvector; add FastAPI health endpoint, initial SQLAlchemy models, environment configuration, and dashboard connectivity.

Exit criteria: Backend health endpoint works; database connects; initial tables exist; React dashboard can read backend status.

Phase 2 — Voice engine (Days 4–8)

Build: Build microphone permissions and audio capture; integrate a Realtime speech-to-speech provider; add modular STT → LLM → TTS adapters; manage sessions, VAD, turn detection, interruptions, and errors.

Exit criteria: User can speak and receive a spoken response; sessions can start/stop safely; latency and failures are logged.

Phase 3 — LLM and agentic core (Days 9–14)

Build: Add OpenAI provider abstraction; implement LangGraph state and orchestration; build intent classification, planning, tool routing, structured tool schemas, retries, and approval gates; implement the first task tools.

Exit criteria: A text request can run through the graph, call a safe tool, return a validated result, and pause for approval when required.

Phase 4 — Tasks and reminders (Days 15–18)

Build: Implement task CRUD, priorities, deadlines, status transitions, reminder scheduling, background worker, and notifications.

Exit criteria: Tasks persist correctly; due reminders trigger; recurring schedules are tested.

Phase 5 — Gmail integration (Days 19–22)

Build: Configure Google OAuth; connect Gmail API; search and retrieve messages; summarize; create drafts; require explicit approval before sending.

Exit criteria: User can find and summarize email and prepare a draft; sending remains approval-gated.

Phase 6 — Google Calendar (Days 23–26)

Build: Configure Calendar OAuth; list events; check availability; detect conflicts; propose and create/reschedule/cancel events with confirmation.

Exit criteria: Calendar reads are correct; write operations require approval and are verified.

Phase 7 — Windows desktop agent (Days 27–32)

Build: Build a local Python companion; pair device securely; establish authenticated outbound WebSocket; add app/browser allowlists, constrained file search, safe file operations, and command validation.

Exit criteria: Paired Windows machine reports status and executes only permitted, validated actions.

Phase 8 — Memory (Days 33–36)

Build: Add conversation state, user preferences, long-term memory, embeddings, pgvector retrieval, and user controls to inspect/correct/delete memories.

Exit criteria: Relevant memory can be retrieved with user isolation and clear retention controls.

Phase 9 — Frontend and UX (Days 37–41)

Build: Build dashboard pages for overview, voice, chat, tasks, calendar, email, reminders, desktop device, approvals, activity, and settings.

Exit criteria: Core workflows can be used from the dashboard; responsive states and error handling are in place.

Phase 10 — Testing and deployment (Days 42–46)

Build: Add unit/integration tests, agent evaluation, security and performance checks, Docker packaging, cloud deployment, logging, monitoring, documentation, and demo recording.

Exit criteria: Reproducible deployment, passing critical tests, documented setup, and a demonstrable end-to-end workflow.

9. Security and reliability requirements

- **Authentication:** protect user-specific APIs and enforce ownership checks on every resource.
- **OAuth:** request only necessary Gmail/Calendar scopes; encrypt refresh tokens; support revocation.
- **Approval policy:** require explicit confirmation for sending email, creating or changing meetings, deleting files, and sensitive desktop/terminal operations.
- **Desktop isolation:** local agent accepts structured commands only; validate action type, arguments, path, and target app against allowlists.
- **Secrets:** keep keys in environment variables or a secrets manager; commit only .env.example; never ask users to paste secrets into chat.
- **Auditability:** record actor, requested action, approval, tool outcome, timestamps, and correlation IDs without logging secrets.
- **Data minimization:** avoid retaining full email bodies or local file contents unless needed and explicitly configured.
- **Resilience:** apply timeouts, retries with limits, idempotency keys for writes, and clear handling of partial failures.

10. Testing and evaluation

Test each specialist tool independently, then test end-to-end workflows. Include adversarial and ambiguous requests, permission failures, timeouts, duplicate events, expired OAuth tokens, and interrupted voice sessions.

Metric	What to measure
Task completion	Did the workflow finish correctly?
Tool accuracy	Were the right tools called with valid arguments?
Approval compliance	Were sensitive actions blocked until approval?
Voice latency	Time from end of user speech to response start
Transcription quality	Word error rate or task-specific transcription accuracy
Reliability	Tool failure rate, retries, timeouts, partial failures
Cost	LLM and voice cost per session/workflow
User experience	Task success, clarification rate, and user feedback

11. Deployment plan

- **Local development:** Docker Compose for PostgreSQL/pgvector and Redis; run FastAPI and Vite locally.
- **Frontend:** deploy React app to Vercel or equivalent.
- **Backend and worker:** deploy FastAPI and background worker to a cloud service or AWS.
- **Database:** managed PostgreSQL with pgvector and automated backups.
- **Desktop:** install the Windows companion locally; it connects outbound to the backend.
- **Operations:** centralized logs, error tracking, health checks, metrics, backups, and documented recovery.

12. Suggested repository structure

```
nexus-ai-assistant/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ api/ # REST and WebSocket routes
■ ■ ■■■ agents/ # task, email, calendar, memory, desktop
■ ■ ■■■ core/ # config, security, policies
■ ■ ■■■ database/ # session, base, migrations
■ ■ ■■■ models/ # SQLAlchemy models
■ ■ ■■■ services/ # OpenAI, Google APIs, voice providers
■ ■ ■■■ workflows/ # LangGraph graphs and state
■ ■■■ tests/
■ ■■■ requirements.txt
■■■ frontend/
■ ■■■ src/
■ ■ ■■■ components/
■ ■ ■■■ pages/
■ ■ ■■■ services/
■ ■ ■■■ hooks/
■ ■■■ package.json
■■■ desktop-agent/
■ ■■■ app/
■ ■■■ policies/
■ ■■■ tests/
■■■ database/
■ ■■■ init/
■■■ docs/
■■■ docker-compose.yml
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

13. Phase 1 starting checklist

Use this as the first implementation checkpoint. Complete the setup locally and verify the outputs before starting the voice engine.

Install and verify tools

Install Python 3.12, Node.js 22 LTS, Git, Docker Desktop (WSL2 backend), and VS Code. Then run the following in PowerShell:

```
python --version
node --version
npm --version
git --version
docker --version
docker compose version
```

Create the repository

```
mkdir nexus-ai-assistant
cd nexus-ai-assistant
mkdir backend frontend desktop-agent docs database
git init
code .
```

Start the database

Create a Docker Compose PostgreSQL service using the pgvector image, expose port 5432 for local development, and enable the vector extension. Verify the container is healthy and connect with psql.

Backend and frontend checkpoint

- Create a Python virtual environment in backend and install FastAPI, Uvicorn, SQLAlchemy, psycopg, Pydantic Settings, python-dotenv, OpenAI SDK, and HTTPX.
- Create a FastAPI app with /health and /api/v1/system/config endpoints.
- Create initial User, Conversation, and Message models.
- Create a React + TypeScript Vite app and connect it to the backend health endpoint.
- Keep secrets in backend/.env; commit a safe .env.example but never commit .env.

Acceptance checks

- Docker reports the database container as healthy.
- PostgreSQL accepts a local connection and the vector extension is installed.
- FastAPI starts and /health reports the database as connected.
- The React dashboard loads and displays backend status.
- Swagger UI opens at /docs.
- The initial database tables are present.

Important implementation notes

- The earlier starter configuration used a placeholder model identifier. Do not assume it is valid; set a verified model name before making an API request.

- Do not share API keys, OAuth tokens, passwords, or SECRET_KEY in chat or screenshots.
- Use SQLAlchemy create_all only for the earliest local prototype; move to Alembic migrations before production.
- The pgvector init SQL runs automatically only when the database volume is first initialized. If the volume already exists, enable the extension manually.
- API usage is billed separately from a ChatGPT subscription.

Working agreement

Proceed one milestone at a time. After Phase 1, share the terminal output or error message (with secrets removed). Diagnose and resolve it before beginning Phase 2. This keeps the implementation reproducible and prevents later features from being built on an unstable foundation.

End of implementation plan